Intermediate XSS Defense - Four-Task Homework

Student Handout - Updated Version with Automated Tests and Manual Browser Verification

1. Assignment Overview

Cross-Site Scripting (XSS) occurs when untrusted data is placed into a web page in a way that causes the browser to interpret it as executable code. In this assignment, students must secure a Python Flask application using context-aware output encoding, URL validation, rich HTML sanitization, browser security headers, and safe DOM update methods.

The final verification has two components: automated `pytest` tests and manual checks in a web browser. Passing the tests provides strong evidence that the implementation satisfies the assignment requirements. However, manual browser verification is still necessary because actual browser behavior matters in defending against XSS.

2. Files Provided to Students

File	Purpose
`incomplete.py`	Starter Flask application with TODO sections. Students must complete this file.
`requirements.txt`	Python dependencies: Flask, nh3, and pytest.
`tests/test_student.py`	Automated tests that import `incomplete.py` and check all four tasks.
This PDF handout	Instructions, verification commands, manual checks, and submission requirements.

Important: Students must not receive an instructor's completed solution file. They should complete only `incomplete.py`.

3. Setup Commands

Windows PowerShell

```
cd xss_4task_student_updated
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

macOS / Linux

```
cd xss_4task_student_updated
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

4. How Students Demonstrate That Their Code Is Correct

Students must demonstrate correctness in two ways:

- **Automated tests:** Run `python -m pytest -q tests/test_student.py` and provide a screenshot showing that all tests passed.
- **Manual browser verification:** Run the Flask application, open the required URLs, confirm that malicious payloads are neutralized, and submit the required screenshots as evidence.

Running the automated tests

```
python -m pytest -q tests/test_student.py
```

Expected result after completing all four tasks:

```
4 passed
```

Before students complete the TODO sections, it is normal for some tests to fail. Those failures indicate which sections still need work.

Running the Flask application for manual verification

```
flask --app incomplete run --debug --port 5000
```

Then open this URL in a browser:

```
http://127.0.0.1:5000/
```

Task 1 - Context-Aware Encoding and Safe URL Validation

What students must implement

Complete these functions in `incomplete.py`:

```
normalize_text()
html_text()
html_attr()
safe_href()
```

Required behavior

- `normalize_text()` must convert input to bounded text, remove control characters, normalize newlines, strip leading/trailing whitespace, and enforce `max_len`.
- `html_text()` must escape text for regular HTML element content. For example, `<b>"x"</b>` should be displayed as visible text, not interpreted as markup.
- `html_attr()` must escape text for quoted HTML attributes, including quotation marks.
- `safe_href()` must allow safe `http`, `https`, and `mailto` URLs and local paths, but reject `javascript:`, `data:`, `vbscript:`, and `file:` URLs, protocol-relative URLs, and URLs containing spaces, quotation marks, angle brackets, or control characters.

Automated test command

```
python -m pytest -q tests/test_student.py::test_task1_context_encoding_and_safe_href
```

Expected result: `1 passed`

Manual browser checks

Search payload: Run the application and open:

```
http://127.0.0.1:5000/search?q=%3Cimg%20src=x%20onerror=alert(1)%3E
```

Expected behavior: The payload appears as text and no `alert` dialog appears. In the page source, dangerous characters must be escaped, for example as `<img`.

Unsafe URL test:

```
http://127.0.0.1:5000/profile?name=Student&website=javascript:alert(1)
```

Expected behavior: The link's `href` value becomes `#`.

Task 1 - Continued

Safe URL test:

```
http://127.0.0.1:5000/profile?name=Student&website=https://example.com
```

Expected behavior: The link remains `https://example.com`.

Deliverables for Task 1

- A screenshot showing that the Task 1 `pytest` test passed.
- A screenshot showing the payload displayed as text on the search page without triggering an alert.
- A screenshot or browser inspector output showing that an unsafe website URL was changed to `href="#"`.
- A screenshot showing that a safe HTTPS URL is still allowed.

Task 2 - Rich HTML Sanitization and Safe Post Rendering

What students must implement

Complete these functions:

```
sanitize_rich_html()
render_post_card()
```

Required behavior

- `sanitize_rich_html()` must allow a small set of safe formatting tags, such as `p`, `strong`, `em`, `ul`, `ol`, `li`, `code`, `pre`, and safe links.
- It must remove dangerous blocks such as `script`, `style`, `iframe`, `object`, `embed`, `svg`, and `math`.
- It must remove unsafe link protocols, such as `javascript:` and `data:`.
- `render_post_card()` must use the correct context for each field: the title as HTML text, author as text and as an attribute, category as an attribute, website as a safe `href`, and body as sanitized rich HTML.

Automated test command

```
python -m pytest -q tests/test_student.py::test_task2_sanitizer_and_safe_post_rendering
```

Expected result: `1 passed`

Manual browser checks

Open `http://127.0.0.1:5000/` and create a post with this body:

```
<p>Hello <strong>safe text</strong></p><script>alert(1)</script>
```

Expected behavior: The `<strong>safe text</strong>` section may remain, but the `<script>` block must be removed and no alert may appear.

Task 2 - Continued

Create another post using unsafe values for the title, author, website, category, and body:

```
Title: <img src=x onerror=alert(1)>
Author: A" onclick="alert(1)
Website: javascript:alert(1)
Category: x" onclick="alert(1)
Body: <p>Normal text</p><script>alert(1)</script>
```

Expected behavior: The title and author are shown as text, the website becomes `href="#"`, and neither an `onclick=` attribute nor a `<script>` tag appears in the rendered card.

Deliverables for Task 2

- A screenshot showing that the Task 2 `pytest` test passed.
- A screenshot showing that permitted rich HTML formatting still works.
- A screenshot showing that script content was removed.
- A screenshot or browser inspector output showing that the unsafe link became `href="#"` and no `onclick` attribute is present.

Task 3 - CSP Nonce, Security Headers, and Safer Cookies

What students must implement

```
create_nonce()
apply_security_headers()
```

Required behavior

- `create_nonce()` must generate a fresh, unpredictable nonce for each response.
- `apply_security_headers()` must add a Content-Security-Policy (CSP) whose `script-src` uses the nonce.
- The CSP must also include `object-src 'none'`, `base-uri 'none'`, and `frame-ancestors 'none'`.
- Response headers must include `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, and `X-Frame-Options: DENY`.
- The demonstration cookie must include `HttpOnly` and `SameSite=Lax`.

Automated test command

```
python -m pytest -q tests/test_student.py::test_task3_security_headers_cookie_and_nonce
```

Expected result: `1 passed`

Manual checks in PowerShell

```
$response = Invoke-WebRequest -Uri "http://127.0.0.1:5000/"
$response.Headers
$response.Headers["Set-Cookie"]
```

Expected behavior: Headers must include `Content-Security-Policy`, `X-Content-Type-Options`, `Referrer-Policy`, and `X-Frame-Options`. The cookie must include `HttpOnly` and `SameSite=Lax`.

Verify the nonce changes for each request:

```
(Invoke-WebRequest -Uri "http://127.0.0.1:5000/").Headers["Content-Security-Policy"]
(Invoke-WebRequest -Uri "http://127.0.0.1:5000/").Headers["Content-Security-Policy"]
```

Expected behavior: The two CSP headers contain different nonce values.

Deliverables for Task 3

- A screenshot showing that the Task 3 `pytest` test passed.
- A screenshot of response headers showing the CSP and security headers.
- A screenshot of `Set-Cookie` showing `HttpOnly` and `SameSite=Lax`.
- A screenshot showing two different CSP nonce values across two requests.

Task 4 - Safe JavaScript State and DOM XSS Defense

What students must implement

```
escape_json_for_html_script()
render_preview_script()
```

Also fix the `/dom` route by replacing unsafe `innerHTML` with `textContent`.

Required behavior

- `escape_json_for_html_script()` must serialize JSON and escape dangerous characters so that `</script>` cannot break out of the script block.
- `render_preview_script()` must use the CSP nonce and write untrusted values using `textContent`, not `innerHTML`.
- The `/dom` route must display the URL hash as text using `textContent`. It must not pass untrusted input to `innerHTML`.

Automated test command

```
python -m pytest -q tests/
test_student.py::test_task4_safe_script_json_and_dom_xss_defense
```

Expected result: `1 passed`

Manual browser checks

Safe preview output:

```
http://127.0.0.1:5000/preview?message=hello
```

Expected behavior: The page displays `hello`.

Script-breakout payload:

```
http://127.0.0.1:5000/preview?message=%3C/script%3E%3Cimg%20src=x%20onerror=alert(1)%3E
```

Expected behavior: The payload is displayed as text, the page is not broken, and no alert appears. In the page source, raw `</script>` must not appear within the JavaScript data.

Task 4 - Continued

DOM XSS test:

```
http://127.0.0.1:5000/dom#%3Cimg%20src=x%20onerror=alert(1)%3E
```

Expected behavior: The payload is displayed as text, no alert appears, and the source code uses `textContent` rather than `innerHTML`.

Deliverables for Task 4

- A screenshot showing that the Task 4 `pytest` test passed.
- A screenshot of the preview page handling the malicious payload safely.
- A screenshot of the `/dom` route displaying the hash payload as text without triggering an alert.
- A screenshot or source-code snippet showing that `textContent` is used for untrusted input instead of `innerHTML`.

5. Final Completion Checklist

- The Flask application runs without Python errors.
- All automated tests pass: `python -m pytest -q tests/test_student.py`.
- The search payload is escaped and does not execute.
- A dangerous profile website URL is converted to `#`.
- Safe `http`, `https`, and local URLs are still permitted.
- Post title, author, category, and `href` fields are safely rendered.
- Rich post bodies remove scripts and unsafe URLs while allowing safe formatting.
- A CSP header is present and includes a fresh nonce.
- Security headers and cookie flags are present.
- The preview page safely handles `</script>` payloads.
- The `/dom` route uses `textContent` and no XSS payload opens an alert dialog.

6. Required Student Submission Items

Deliverable	Description
Completed `incomplete.py`	The only Python application file students should submit.
Automated-test evidence	A screenshot showing that all tests passed with `python -m pytest -q tests/test_student.py`.
Manual browser screenshots	Screenshots for Tasks 1-4 demonstrating safe behavior and the absence of alerts.
Header evidence	A screenshot from PowerShell or browser developer tools showing the CSP, security headers, and cookie flags.
Short report	A brief explanation of what changed in each task and why those changes help prevent XSS.

7. What the Automated Tests Prove

The automated tests check whether the required Python functions and Flask routes behave correctly for the specified cases. Passing all tests provides strong evidence that the code meets the assignment requirements. Nevertheless, students must also perform manual checks in a browser because XSS ultimately depends on how the browser interprets HTML, JavaScript, URLs, cookies, and DOM operations.

8. Further Reading

- **OWASP Cross Site Scripting Prevention Cheat Sheet:** Context-aware output encoding, sanitization, and defense in depth.
- **MDN documentation:** `innerHTML` and the risks of client-side XSS.
- **pytest documentation:** Discovering test files named `test_*.py` and running tests directly.

Appendix - How to Use the Student Test File

`tests/test_student.py` imports only `incomplete.py`. It does not import or require any completed solution file. Students can run the complete test file with:

```
python -m pytest -q tests/test_student.py
```

They can also run each task separately:

```
python -m pytest -q tests/test_student.py::test_task1_context_encoding_and_safe_href
python -m pytest -q tests/test_student.py::test_task2_sanitizer_and_safe_post_rendering
python -m pytest -q tests/test_student.py::test_task3_security_headers_cookie_and_nonce
python -m pytest -q tests/
test_student.py::test_task4_safe_script_json_and_dom_xss_defense
```

Below is a simplified excerpt of the test file. The complete version is provided separately as `tests/test_student.py`.

```
import incomplete as student

PAYLOAD = '<img src=x onerror=alert(1)>'

def test_task1_context_encoding_and_safe_href():
    assert student.html_text('<b>"x"</b>') == '&lt;b&gt;"x"&lt;/b&gt;'
    assert student.safe_href('javascript:alert(1)') == '#'
    assert student.safe_href('https://example.com/profile') == 'https://example.com/
profile'

def test_task4_safe_script_json_and_dom_xss_defense():
    state = {'message': '</script><img src=x onerror=alert(1)>'}
    safe_json = student.escape_json_for_html_script(state)
    assert '</script>' not in safe_json.lower()
    assert '\\u003c/script\\u003e' in safe_json.lower()
    script = student.render_preview_script(state, 'abc123')
    assert 'textContent' in script
    assert '.innerHTML' not in script
```